Validation & Test Engineering Report

Argo CD Agent Integration (Managed Mode) Verification Matrix across Multi-Cluster Topologies

1. Executive Test Summary

This formal engineering document outlines the structural validation and results matrix for the **Argo CD Agent Integration (Managed Mode)** utilizing the `gitops-addon` layer across heterogeneous kubernetes environments. The testing architecture coordinates a central Red Hat Advanced Cluster Management (RHACM) Hub cluster orchestrating two managed downstream environments: a production-grade **Red Hat OpenShift Container Platform (OCP)** cluster and a localized upstream **Kind** cluster.

The primary objective is confirming seamless **Advanced Pull Model** topology execution, validating decentralized multi-cluster reconciliation via mutual TLS (mTLS), auditing role-based access control propagation, and evaluating strict, zero-intervention lifecycle deletion boundaries.

8 / 8	2	100%	0
SCENARIOS PASSED	TARGET ENVIRONMENTS	MANIFEST COMPLIANCE	MANUAL PATCHES DEFECTED

> **Key Engineering Architectural Improvement Summary**
>
> To ensure production stability, manual out-of-band policy patching loops have been eliminated. This configuration relies completely on decoupled, declarative companion configurations and dynamic OCM-driven `ApplicationSets` to prevent state drift or reconciliation collisions on the Hub.

2. Updated Production Manifests & Pipeline Strategy

Phase 1: Hub Cluster Principal Configuration

Configuring the Hub's Argo CD instance as the Principal authority disables local workload synchronization execution threads and enables incoming secure mTLS handshakes from remote spoke agents.

```
export KUBECONFIG=/path/to/hub/kubeconfig

kubectl patch argocd openshift-gitops -n openshift-gitops --type=merge -p '{
  "spec": {
    "controller": { "enabled": false },
    "argoCDAgent": {
      "principal": {
        "enabled": true,
        "auth": "mtls:CN=system:open-cluster-management:cluster:([^:]+):addon:gitops-
addon:agent:gitops-addon-agent",
        "namespace": { "allowedNamespaces": ["*"] },
        "server": { "route": { "enabled": true } }
```

```
      }
    },
    "sourceNamespaces": ["*"]
  }
}'
```

Phase 2: Add-On Placement and Stable Companion Policy Setup

The placement defines the structural selection scope target matrix. Instead of applying unstable manual inline edits to the controller-generated policy, a decoupled `PlacementBinding` couples a dedicated companion configuration policy to push downstream cluster RBAC and initial namespaces without drifting.

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: agent-placement
  namespace: openshift-gitops
spec:
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchExpressions:
            - key: name
              operator: In
              values: ["ocp-cluster1", "kind-cluster1"]
---
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
metadata:
  name: agent-gitops
  namespace: openshift-gitops
spec:
  argoServer: { "cluster": "local-cluster" }
  argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: agent-placement
  gitopsAddon: { "enabled": true }
  argoCDAgent: { "enabled": true, "mode": "managed" }
```

```yaml
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: agent-custom-spoke-setup
  namespace: openshift-gitops
spec:
  reremediationAction: enforce
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: agent-spoke-rbac-and-namespace
        spec:
          reremediationAction: enforce
          severity: medium
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: rbac.authorization.k8s.io/v1
                kind: ClusterRoleBinding
                metadata:
                  name: acm-openshift-gitops-cluster-admin
                roleRef:
                  apiGroup: rbac.authorization.k8s.io
                  kind: ClusterRole
                  name: cluster-admin
                subjects:
                  - kind: ServiceAccount
                    name: acm-openshift-gitops-argocd-application-controller
                    namespace: openshift-gitops
            - complianceType: musthave
              objectDefinition:
                apiVersion: v1
                kind: Namespace
                metadata:
                  name: guestbook
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: agent-custom-setup-binding
  namespace: openshift-gitops
placementRef:
  name: agent-placement
  kind: Placement
  apiGroup: cluster.open-cluster-management.io
subjects:
```

```
  - name: agent-custom-spoke-setup
    kind: Policy
    apiGroup: policy.open-cluster-management.io
```

Phase 3: Automated ApplicationSet Configuration

Using destination-based routing (`destination.name`) paired with an automated multi-cluster decision generator avoids duplication and automatically targets registered environments.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: guestbook-agent-deployment
  namespace: openshift-gitops
spec:
  generators:
    - clusterDecisionResource:
        configMapRef: acm-placement
        labelSelector:
          matchLabels:
            cluster.open-cluster-management.io/placement: agent-placement
        requeueAfterSeconds: 180
  template:
    metadata:
      name: 'guestbook-{{name}}'
    spec:
      project: default
      source:
        repoURL: https://github.com/argoproj/argocd-example-apps
        targetRevision: HEAD
        path: guestbook
      destination:
        name: '{{name}}'
        namespace: guestbook
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
```

3. Formal Execution & Test Results Matrix

The following matrix compiles the formal execution outcomes observed during the test cycle. Assertions were evaluated under a strict read-only execution profile on the spoke layers unless checking standard teardown pathways.

Test ID	Scope / Context	Assertion / Verification Condition	Status & Evidence
TC-01	Hub Principal	Validate local controller deactivation and mTLS listener initialization patterns via specific logs.	**PASSED** mTLS handshakes verified on Principal logs.
TC-02	Hub Control	Confirm `managedclusteraddon` generation and check cluster proxy secret generation mapping parameters.	**PASSED** Secret endpoints populate mapping structures successfully.
TC-03	OCP Spoke	Verify OLM auto-detection loop configures `DISABLE_DEFAULT_ARGOCD_INSTANCE=true` systematically.	**PASSED** Subscription env validation verified.
TC-04	OCP Workload	Audit deployment placement sync. Ensure expected container failure occurs due to OpenShift SCC restricted-v2 security boundaries (Port 80 binding lock).	**EXPECTED FAILURE** Deployment successfully synced; SCC block active as intended.
TC-05	Kind Spoke	Verify non-OCP context safety fallback. Confirm OLM subscription skipped and native chart templates applied.	**PASSED** Deployment completely functional on localized container topology.
TC-06	Kind Images	Audit image source declarations. Verify standard Red Hat enterprise registry fallback to external public nodes.	**PASSED** Image references resolved cleanly without pull timeouts.
TC-07	Teardown Lifecycle	Execute structured cascading finalizer validation (ApplicationSet → GitOpsCluster → Policies → Placements).	**PASSED** Zero finalizer hangs encountered; zero manual intervention required.
TC-08	Spoke Post-Cleanup	Ensure target namespaces and customer user resources stay isolated on spokes while infrastructure add-ons are swept away.	**VERIFIED** User data preserved; Core system operators removed clean.

4. Production Deletion Sequence & Verification

To prevent deadlocks or orphaned objects on target spokes, cleanup sequences must proceed down explicit resource hierarchies. Finalizers clear natively without manual `kubectl edit` interventions.

```
# Step 1: Remove workload application objects first to allow clean un-sync
kubectl delete applicationset guestbook-agent-deployment -n openshift-gitops

# Step 2: Remove central GitOps manager configuration
kubectl delete gitopscluster agent-gitops -n openshift-gitops

# Step 3: Tear down custom infrastructure policies and tracking markers
kubectl delete policy agent-custom-spoke-setup -n openshift-gitops
kubectl delete placementbinding agent-custom-setup-binding -n openshift-gitops

# Step 4: Revoke base selection placements
kubectl delete placement agent-placement -n openshift-gitops
```

Report Conclusion: All automated and manual integration test scenarios have run successfully. The architecture matches enterprise operational resilience requirements.